

Yes. For **today's work**, I would separate commands into “**memorize**”, “**understand but look up when needed**”, and “**rarely used.**”

Since your goal is to become capable without AI, don't try to memorize everything. A professional developer memorizes the **high-frequency workflow**, not every possible Git command.

Day 1 — Commands Worth Memorizing

1. Python / Virtual Environment

Check Python version

Bash

```
python3 --version
```

Meaning: What version of Python am I using?

Create a virtual environment

Bash

```
python3 -m venv .venv
```

Meaning: Create a Python environment called `.venv` for this project.

 **Memorize this.** You'll use it for almost every new Python project.

Activate virtual environment — Mac/Linux

Bash

```
source .venv/bin/activate
```

Meaning: Start using this project's Python environment.

 **Memorize this.**

You'll then see:

```
(.venv)
```

in your terminal.

Deactivate

```
Bash
deactivate
```

Meaning: Leave the virtual environment.

📌 Worth memorizing, although you'll probably use it less often.

Check which Python you're using

```
Bash
which python3
```

Meaning: Show me the actual Python executable being used.

Useful when something seems wrong.

2. Running Python

Run a Python file

```
Bash
python3 hello.py
```

Meaning:

Run `hello.py` using Python.

📌 **Definitely memorize this.**

Later you'll use:

```
Bash
python3 app.py
python3 main.py
```

```
python3 test.py
```

etc.

3. Git — The Most Important Commands

These are the commands I want you to become comfortable with.

Start Git in a project

Bash

```
git init
```

Mental shortcut:

`init` = initialize/start

📌 Memorize.

You normally do this **once per project**.

Check what's happening

Bash

```
git status
```

Mental shortcut:

`status` = "What's going on?"

📌 **VERY important. Memorize.**

You'll use this constantly.

Stage changes

Bash

```
git add .
```

Mental shortcut:

| add = prepare changes for the next commit

📌 Memorize.

The `.` means:

| everything in this folder that isn't ignored.

Save a version

Bash

```
git commit -m "message"
```

For example:

Bash

```
git commit -m "Add Python basics"
```

Mental shortcut:

| commit = save a snapshot/version

📌 Memorize.

Upload to GitHub

Bash

```
git push
```

Mental shortcut:

| push = send my commits to GitHub

📌 Definitely memorize.

Download changes from GitHub

```
Bash
```

```
git pull
```

Mental shortcut:

| pull = bring changes from GitHub to my computer

📌 Memorize.

4. Git History

See previous commits

```
Bash
```

```
git log --oneline
```

Mental shortcut:

| log = history

Example:

```
2c6e113 Initial Python project
```

Useful, but **not as important as** `status`, `add`, `commit`, `push`, `pull`.

5. Commands You Should Understand, But Don't Need to Memorize Yet

We used:

```
Bash
```

```
git remote -v
```

Understand:

| Show me the GitHub repositories connected to this project.

But don't worry if you forget the exact syntax.

We also used:

Bash

```
git remote add origin URL
```

Understand the concept:

```
remote = online repository  
origin = nickname for that repository
```

But this is something you can easily look up.

You normally do it **once when connecting a project to GitHub**.

We also used:

Bash

```
git branch -M main
```

Understand:

Rename/set the current branch as `main`.

Don't prioritize memorizing this yet.

★ Your Professional "Core 15"

If I were teaching you from zero and wanted you to eventually work independently, I'd put these on your cheat sheet:

Bash

```
# Python  
python3 --version  
python3 -m venv .venv  
source .venv/bin/activate  
deactivate  
python3 filename.py  
  
# Git  
git init  
git status  
git add .
```

```
git commit -m "message"
git push
git pull
git log --oneline

# Useful checks
which python3
git remote -v
```

But there's an even smaller set I want you to **actively memorize first**:

```
python3 filename.py

python3 -m venv .venv
source .venv/bin/activate

git status
git add .
git commit -m "message"
git push
git pull
```

Your mental dictionary

Try to associate:

```
venv      → Python environment
activate   → enter environment
deactivate → leave environment

status    → check
add        → prepare
commit     → save
push       → upload
pull       → download
log        → history
```

If you know **what each command is trying to accomplish**, you're learning like a developer—not just memorizing syntax.

And don't worry about the commands you forget. **Looking up syntax is normal even for experienced developers.** The thing you should build is the ability to know *what you need to do next*.